

Exercise 1: Read-Only Code Investigation (Live) — 20 minutes

Scenario

The Atlas Inventory dashboard (`http://localhost:8080`) shows three summary numbers above the table: **items**, **units**, and **zones**. Before anyone is allowed to change this code, you need to understand exactly how those numbers are produced.

Your task: identify where the summary is calculated, determine how a "zone" is derived from an inventory item, and explain which tests currently cover that calculation. **Do not modify any files.**

Prompt template

```
we need to understand the inventory dashboard summary (item count, unit count, zone count) shown above the table.
```

```
inspect only the files needed to answer these questions:
```

1. where is the summary calculated?
2. how is a "zone" derived from an inventory item?
3. which tests currently cover this calculation?
4. what is one likely edge case that is not tested?

```
do not edit files or run commands that modify the repository. keep the answer under 300 words and include file names and symbols.
```

Paste this into OpenCode as-is, without additional context up front. Part of the exercise is watching which files the agent decides to open/look into.

What you should expect

- The calculation lives in `renderSummary()` in `src/main/resources/static/app.js`.

- A zone is derived by splitting `storageLocation` on `-` and taking the first segment (for example `"A-01-01"` → zone `"A"`).
- There is no automated test file for `app.js` anywhere in `src/test/` . The only tests that touch this data path are the Java integration tests in `InventoryServerTest.java` , and none of them assert on the summary numbers — they only check the API responses and the `/health` endpoint.
- A reasonable edge case: a `storageLocation` value with no `-` in it (or an empty string) still produces a "zone", but it will be the whole location string, which may not be the intended behavior.

Learnings

- Ask for investigation before implementation, don't let agents jump straight to a fix.
- Limit the number of files and the size of the output so the answer stays reviewable.
- Require file names and symbol names in the answer so claims can be checked.
- Model outputs are not facts. Treat any answer as a hypothesis to verify. Open the files yourself and confirm the agent is right.
- Correct the agent if it starts exploring irrelevant parts of the repository, such as `InventoryRepository.java` or the database schema, which have nothing to do with the dashboard summary.

Debrief question

"What did the agent know because it inspected the repository, and what did it merely assume?"